

# SMARTFACTORY

## Complete Build Blueprint

Digital Twin + Predictive Intelligence + Autonomous Factory Orchestration

| SOURCE OF TRUTH                                      | LOCKED IMPLEMENTATION                   |
|------------------------------------------------------|-----------------------------------------|
| AI-Powered Smart Factory project document            | HTML5 Canvas + Vanilla JS               |
| Six connected control domains                        | Python + FastAPI + WebSocket            |
| SENSE → UNDERSTAND → PREDICT → DECIDE → ACT → UPDATE | Market-Based Auction Engine             |
| Connected failure propagation                        | SQLite + Discrete-Event Simulation + A* |

### Purpose

This document combines the project requirements, dashboard/UI decisions, locked system architecture, implementation rules, demo flow, data model, algorithms, APIs, testing plan, and a single master prompt that can be given to an AI coding agent to build the prototype from scratch.

Important: the architecture in this document overrides earlier exploratory suggestions. Use the locked stack below.

# 1. Project Definition

Project: AI-Powered Smart Factory Digital Twin — a live virtual representation of a factory that detects abnormal conditions, predicts operational risks, understands cross-system impact, and coordinates machines, materials, AMRs/AGVs, production and human intervention.

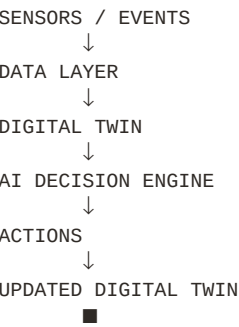
Core problem: factory resources are interconnected. A local disruption can propagate into production, inventory, transport, schedules and human tasks. The system therefore needs one continuously updated decision layer rather than isolated failure handlers.

Core loop: SENSE → UNDERSTAND → PREDICT → DECIDE → ACT → UPDATE.

Prototype positioning: this is a hackathon decision-support/orchestration prototype, not a certified industrial safety controller. Do not claim that autonomous transport, fleet management or dynamic routing were invented by the project.

# 2. Locked System Architecture

| Layer      | Technology                        | Responsibility                                                                               |
|------------|-----------------------------------|----------------------------------------------------------------------------------------------|
| Frontend   | HTML5 Canvas + Vanilla JavaScript | Live factory dashboard, real-time controls, factory simulation, fleet, auction and telemetry |
| Backend    | Python + FastAPI                  | REST APIs, WebSocket state streaming, event processing and asynchronous simulation tick loop |
| AI Layer   | Market-Based Auction Engine       | Multi-attribute AGV bidding and future-impact congestion heuristic                           |
| Database   | SQLite                            | Task history, auction bids, completion times and throughput analytics                        |
| Simulation | Discrete-Event Tick Loop          | A* routing, AGV movement, congestion, battery cycles, failures and recovery                  |



# 3. Six Control Domains

| Domain        | Controls / Events                                                   | Typical response                                              |
|---------------|---------------------------------------------------------------------|---------------------------------------------------------------|
| 01. Machine   | Breakdown, temperature, vibration, speed drop, maintenance          | Reduce load, reassign production, schedule maintenance        |
| 02. Material  | Low stock, shortage, wrong material, delivery delay, warehouse full | Predict demand, replenish, reroute, quarantine wrong material |
| 03. AMR / AGV | Low battery, failure, stuck, congestion, blocked route              | Charge, recover, reassign task, dynamically reroute           |

| Domain         | Controls / Events                                              | Typical response                                              |
|----------------|----------------------------------------------------------------|---------------------------------------------------------------|
| 04. Production | Bottleneck, delay, urgent order, quality defect, demand change | Redistribute work, reprioritize, isolate defects, reschedule  |
| 05. Factory    | Power, network, emergency, environment, security               | Safe state, isolate affected area, controlled recovery, alert |
| 06. Human      | Worker unavailable, task error, safety issue, intervention     | Reassign, validate, stop affected operation, escalate         |

These are six connected control areas, not six independent mini-projects.

## 4. Final Dashboard / UI Specification

The UI should be clean, spacious and judge-friendly. Use a warm cream base, brown for structure/headings, restrained orange for attention states, and red/orange primarily for alerts and critical messages. Avoid the congested dark reference look; use the reference only for information hierarchy.

Overall layout: three columns. Left = six-domain controls + event stream. Center = entire factory simulation + AMR/AGV fleet below. Right = live AGV auction + selected task/decision trace + system status/KPIs.

| Area          | Required content                                                                                                                            |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Top bar       | SmartFactory title, Live Simulation, system connection state, clock, compact SENSE → UNDERSTAND → PREDICT → DECIDE → ACT → UPDATE indicator |
| Left          | Exactly six domain control cards, each expandable; Event Stream below them                                                                  |
| Center top    | Interactive factory floor digital twin rendered on HTML5 Canvas                                                                             |
| Center bottom | AMR/AGV Fleet summary cards + live fleet table                                                                                              |
| Right top     | AGV Live Auction with task, bids, travel, battery, congestion, future impact and final bid                                                  |
| Right middle  | Selected Task / Decision Trace                                                                                                              |
| Right bottom  | System Status + live KPIs; alert styling only when needed                                                                                   |

## 5. Factory Simulation Requirements

- Represent raw material/storage, machines, production stations, quality inspection, finished goods, dispatch, charging stations, corridors, obstacles and restricted/blocked zones.
- Show multiple AGVs/AMRs as moving entities with ID, status, route, battery and task.
- Render route traces, destinations, charging locations, congestion indicators and blocked areas.
- Support zoom/pan and clicking an AGV or factory component to inspect it.
- Simulation is live, not a static illustration. AGV positions and states must update from the backend tick stream.
- When a route is blocked, the affected route visually changes and the planner searches for an alternative.

## 6. AMR / AGV Fleet

| AGV    | Status | Task     | Location   | Battery | Speed   |
|--------|--------|----------|------------|---------|---------|
| AGV-01 | Moving | TASK-108 | Machine-02 | 82%     | 1.2 m/s |

| AGV    | Status   | Task     | Location   | Battery | Speed |
|--------|----------|----------|------------|---------|-------|
| AGV-02 | Bidding  | TASK-109 | Storage-S1 | 64%     | 0     |
| AGV-03 | Charging | —        | Charger-C1 | 31%     | 0     |
| AGV-04 | Idle     | —        | Station-B  | 91%     | 0     |

Summary cards: Total AMRs | Active | Idle | Charging | Failed.

## 7. Live AGV Auction

The auction is the visible implementation of the AI layer. A task should not be assigned simply to the nearest AGV. Each candidate evaluates multiple attributes and the engine chooses the bid using the configured market/heuristic policy.

| AGV    | Travel | Battery | Congestion | Future Impact | Final Bid |
|--------|--------|---------|------------|---------------|-----------|
| AGV-02 | 18 s   | 72%     | 0.2        | 1.1           | 21.4      |
| AGV-05 | 22 s   | 88%     | 0.1        | 0.6           | 20.8      |
| AGV-03 | 16 s   | 43%     | 0.7        | 2.3           | 24.6      |

The UI should make the trade-off understandable: the winning bid is a function of current travel cost plus resource and future factory impact, not nearest-distance alone.

## 8. Connected Failure Propagation — Core Demonstration

Every injected event must enter the same digital twin/event pipeline. The system should automatically calculate affected entities and produce a recovery action. The judge creates the problem; the system demonstrates the response.

```

Example:
Machine-03 vibration increases
  ↓
Machine health risk rises
  ↓
Affected production jobs identified
  ↓
Future material demand changes
  ↓
AMR transport task changes
  ↓
Auction/re-auction starts
  ↓
AGV bids using travel + battery + congestion + future impact
  ↓
A* route selected
  ↓
AGV moves
  ↓
Inventory + production + metrics update
  ↓
Digital twin reaches recovered state

```

Required visual effect: the machine, event stream, affected production state, AGV behavior, auction, and metrics must change as one connected story.

## 9. Required Demo Scenarios

| Scenario            | Trigger                        | Expected visible chain                                                                                            |
|---------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Machine degradation | Machine-03 high vibration      | Risk rises → workload changes → production/AMR impact → reallocation → metrics update                             |
| AMR failure         | Fail AGV-02                    | AGV becomes failed → task removed from allocation → re-auction → another AGV gets task → route updates            |
| Blocked route       | Block corridor                 | Route marked blocked → affected AGVs/tasks identified → A* alternate route → congestion changes                   |
| Material shortage   | Low stock / shortage           | Inventory changes → affected production task identified → replenishment transport task → auction → stock recovery |
| Urgent order        | Create urgent production order | Priority changes → resources evaluated → transport task created → auction → execution                             |

## 10. Automation Rules by Domain

### Machine

- Breakdown: mark failed, identify affected orders/material/AMR tasks, find compatible capacity, reassign work, update schedule.
- High temperature/vibration: calculate risk from current/trend data, reduce or redistribute workload, create maintenance recommendation.
- Speed drop: detect cycle-time/output degradation, identify bottleneck, reallocate suitable work.
- Maintenance: avoid new assignments and schedule safe maintenance.

### Material

- Low stock: track consumption and forecast critical level; create replenishment task.
- Shortage: identify affected jobs, search alternative source, reschedule if required.
- Wrong material: virtual RFID/barcode verification; block task and quarantine/return wrong material.
- Delivery delay: compare expected/actual progress and recalculate transport.
- Warehouse full: redirect incoming material or finished goods to available storage.

### AMR / AGV

- Low battery: decide whether vehicle can complete task; finish-and-charge or transfer task.
- Failure: remove from active allocation and reassign unfinished work.
- Stuck: detect unexpected stop; attempt recovery/alternate route, then transfer if needed.
- Congestion/blocked route: recalculate traffic and route using distance, congestion, battery, destination and priority.

### Production

- Bottleneck: detect high queue/utilization/slow process and redistribute suitable work.
- Delay: compare expected vs predicted completion and reprioritize.
- Urgent order: check material/machine/AMR availability and dynamically allocate resources.
- Quality defect: isolate batch and correlate machine/material/process information.
- Demand change: recalculate material requirements, capacity and transport tasks.

### Factory

- Power: place affected equipment/AMRs in safe simulation states, preserve state, controlled recovery.
- Network: prevent unsafe new commands and synchronize after reconnection.
- Emergency: safety priority, restrict affected zones/routes, alert operator.
- Environmental/security anomaly: protect affected process/zone and escalate.

## Human

- Worker unavailable: identify assigned tasks and find qualified replacement or automatable alternative.
- Task error: validate intended vs actual action; contain and request correction.
- Safety violation: detect restricted-zone entry; stop affected autonomous movement and alert.
- Manual intervention: if confidence is low or risk is high, hold task and request human decision.

# 11. Backend Architecture

```

backend/
  app/
    main.py
    config.py
  api/
    routes_factory.py
    routes_agv.py
    routes_tasks.py
    routes_auction.py
    routes_events.py
    routes_controls.py
  websocket/
    manager.py
    protocol.py
  digital_twin/
    state.py
    event_bus.py
    propagation.py
    rules.py
  simulation/
    engine.py
    clock.py
    world.py
    movement.py
    battery.py
    congestion.py
    failures.py
    recovery.py
  routing/
    astar.py
    graph.py
  auction/
    engine.py
    bidder.py
    scoring.py
    future_impact.py
  database/
    connection.py
    models.py
    repository.py
  services/
    task_service.py
    inventory_service.py
    production_service.py
    metrics_service.py
    event_service.py
  tests/

```

FastAPI responsibilities: initialize state, expose REST endpoints, accept control events, run the async tick loop, invoke auction/routing/recovery logic, persist history to SQLite, and broadcast authoritative state over WebSocket.

## 12. Simulation Tick Loop

Every tick:

1. Read/advance virtual sensor values.
2. Update machine health and production state.
3. Update inventory consumption/transfers.
4. Update AGV positions and battery.
5. Detect stuck vehicles and congestion.
6. Detect newly blocked/failed resources.
7. Propagate event impacts through the digital twin.
8. Generate/revise transport tasks.
9. Run auctions when assignment is required.
10. Run A\* route planning.
11. Apply recovery actions.
12. Recalculate KPIs.
13. Persist important events/history.
14. Broadcast a compact WebSocket state update.

Use a deterministic simulation mode for demos so the same scenario produces repeatable behavior.

Also provide a normal continuous mode for a more natural live simulation.

## 13. Core Data Model

| Entity          | Important fields                                                                          |
|-----------------|-------------------------------------------------------------------------------------------|
| Machine         | id, name, state, temperature, vibration, speed, healthRisk, maintenanceState, position    |
| Material        | id, name, stock, reserved, capacity, reorderPoint, location, expectedIncoming             |
| AGV             | id, status, battery, position, heading, currentTask, speed, route, capability, lastUpdate |
| Task            | id, type, pickup, dropoff, priority, material, deadline, status, assignedAgv, createdAt   |
| Auction         | id, taskId, status, startedAt, closesAt, winner, policy                                   |
| Bid             | id, auctionId, agvId, travelCost, batteryCost, congestionCost, futureImpact, finalBid     |
| ProductionOrder | id, product, quantity, priority, dueAt, machine, status, materialRequirements             |
| Worker          | id, name, zone, availability, qualification, task, safetyState                            |
| Event           | id, timestamp, domain, type, severity, targetId, message, source                          |
| Route           | id, agvId, nodes, distance, estimatedTime, congestion, blocked, createdAt                 |
| Metric          | timestamp, throughput, utilization, avgTaskTime, distance, delays, congestionEvents       |

## 14. WebSocket Event Contract

```
{
  "type": "AGV_UPDATE",
  "timestamp": "2026-09-19T10:15:00Z",
  "data": {
    "id": "AGV-05",
    "position": {"x": 12.7, "y": 5.3},
    "status": "MOVING",
    "battery": 76,
    "task": "TASK-108"
  }
}

{
  "type": "NEW_EVENT",
  "data": {
    "id": "EVT-502",
    "domain": "Machine",
    "severity": "WARNING",
    "message": "Machine-03 vibration above threshold"
  }
}
```

```

}

{
  "type": "AUCTION_UPDATE",
  "data": {
    "taskId": "TASK-108",
    "status": "BIDDING",
    "bids": []
  }
}

{
  "type": "DECISION_TRACE",
  "data": {
    "stage": "DECIDE",
    "message": "AGV-05 selected after future-impact evaluation"
  }
}

```

## 15. REST API Contract

| Method | Endpoint                      | Purpose                                         |
|--------|-------------------------------|-------------------------------------------------|
| GET    | /api/state                    | Return current complete digital-twin snapshot   |
| GET    | /api/agvs                     | Current fleet state                             |
| GET    | /api/tasks                    | Tasks with optional status filter               |
| GET    | /api/events                   | Recent event history                            |
| GET    | /api/auctions                 | Current/recent auctions                         |
| POST   | /api/events/inject            | Inject a domain event/failure                   |
| POST   | /api/tasks                    | Create transport/production task                |
| POST   | /api/auctions/{taskId}/start  | Start live auction                              |
| POST   | /api/auctions/{auctionId}/bid | Submit bid                                      |
| POST   | /api/controls/{domain}        | Execute a control action                        |
| POST   | /api/simulation/reset         | Reset deterministic demo state                  |
| POST   | /api/simulation/pause         | Pause/resume simulation                         |
| GET    | /api/metrics                  | Current and historical KPIs                     |
| WS     | /ws                           | Push live state/events/auction/decision updates |

## 16. Frontend Structure — Vanilla JavaScript

```

frontend/
  index.html
  css/
    base.css
    layout.css
    components.css
    simulation.css
    alerts.css
  js/
    app.js
    api.js
    websocket.js
    state.js
    events.js
    controls.js
    simulation/
      canvas.js

```

```

world.js
renderer.js
agv.js
routes.js
interactions.js
fleet.js
auction.js
decision-trace.js
metrics.js
ui.js
formatters.js
assets/
  icons/
  fonts/

```

Use semantic HTML and plain JavaScript modules. Keep the UI framework-free. HTML5 Canvas is the authoritative renderer for the factory floor. CSS handles layout/cards/tables; JavaScript handles state, controls, WebSocket events and simulation rendering.

## 17. Visual Design System

| Token            | Recommended use                                      |
|------------------|------------------------------------------------------|
| Cream background | #F6F0E7 — page background                            |
| Warm white       | #FFFDFC — cards/panels                               |
| Dark brown       | #4B3022 — headings, primary structure                |
| Brown            | #76513C — secondary text/borders                     |
| Orange           | #D97706 — attention, active states, auction emphasis |
| Red              | #C62828 — failure, emergency, critical alerts only   |
| Green            | #3F7D4A — healthy/normal states                      |

Avoid rainbow status colors. Normal UI should remain calm; red/orange should visually mean that something needs attention.

Use generous spacing, consistent 8px/4px spacing increments, rounded cards, clear section headers, compact tables, and restrained shadows. Avoid dense gradients, excessive animations, giant icons, or decorative elements that compete with the simulation.

## 18. Decision Trace

```

SENSE
Machine-03 vibration detected
↓
UNDERSTAND
Machine health risk increased
↓
PREDICT
Production + transport impact estimated
↓
DECIDE
Reduce load and reassign compatible work
↓
ACT
Re-auction transport task and reroute AGV
↓
UPDATE

```

Digital twin, inventory, production and KPIs synchronized  
 This compact trace should be visible in the right panel and update live. It is a key judge-facing explanation of the intelligence layer.

## 19. Safety, Security and Reliability Rules

- Treat emergency and safety events as highest-priority states in the simulation.
- Never represent the prototype as a certified industrial safety controller.
- Validate all control inputs on the FastAPI server; do not trust browser state.
- Use HTTPS/WSS in deployed environments; keep local development simple.
- Restrict CORS to known origins in deployment.
- Store audit events for manual controls and AI decisions.
- Use rate limiting on sensitive endpoints if exposed beyond a local hackathon environment.
- Keep server-side digital-twin state authoritative; the browser is a visualization/control client.
- Use deterministic scenario IDs and reset functionality so judges can reproduce demos.

## 20. Testing Plan

| Test level  | Must verify                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------------------|
| Unit        | A* pathfinding, bid calculation, future-impact score, battery logic, congestion detection, event propagation |
| Integration | Control event → digital twin → AI → task/route → WebSocket → UI                                              |
| Scenario    | Machine vibration, AGV failure, blocked route, material shortage, urgent order                               |
| UI/E2E      | Six controls, event stream, canvas state changes, auction updates, reset/pause, fleet table                  |
| Determinism | Same seed + same injected event produces repeatable result                                                   |
| Performance | Smooth canvas rendering and timely WebSocket updates for the prototype fleet                                 |

## 21. Recommended Build Order

| Phase | Build first                                                                     |
|-------|---------------------------------------------------------------------------------|
| 1     | Project folders, FastAPI app, SQLite connection, base HTML/CSS/JS shell         |
| 2     | Digital-twin state model and seeded factory world                               |
| 3     | HTML5 Canvas factory renderer with machines, storage, routes, chargers and AGVs |
| 4     | Simulation tick loop + AGV movement + battery                                   |
| 5     | A* route planner + blocked corridor + congestion                                |
| 6     | Six-domain control panel + event injection                                      |
| 7     | Event propagation engine                                                        |
| 8     | Task generation + market auction + multi-attribute bids                         |
| 9     | WebSocket live updates + event stream + fleet                                   |
| 10    | Decision trace + KPIs + recovery scenarios                                      |
| 11    | Polish, deterministic demo mode, testing and deployment                         |

## 22. MASTER PROMPT — GIVE THIS TO YOUR AI CODING AGENT

Copy the following prompt as one instruction to the coding agent. It intentionally locks the architecture so the agent does not silently substitute React, Node.js, MongoDB or another stack.

### BEGIN MASTER PROMPT

You are a senior full-stack engineer, simulation engineer, optimization engineer, and UI/UX engineer. Build the SmartFactory project completely from scratch as a working hackathon prototype.

#### PROJECT PURPOSE

Build an AI-powered Smart Factory Digital Twin that represents a factory in real time, detects abnormal conditions, predicts operational risks, understands cross-system impact, dynamically assigns material-transport tasks to AMRs/AGVs, and coordinates recovery across machines, materials, production, factory conditions and humans.

#### CORE LOOP

SENSE → UNDERSTAND → PREDICT → DECIDE → ACT → UPDATE

#### LOCKED ARCHITECTURE — DO NOT SUBSTITUTE

1. Frontend: HTML5 Canvas + Vanilla JavaScript + HTML + CSS.
2. Backend: Python + FastAPI.
3. AI layer: Market-Based Auction Engine using multi-attribute bidding and a future-impact congestion heuristic.
4. Database: SQLite.
5. Simulation: Discrete-Event Tick Loop with A\* congestion-aware routing, battery cycles, failure handling and recovery.

Do NOT replace this with React, Vue, Svelte, Node.js, MongoDB, Firebase or another architecture.

#### PROJECT RULE

This must behave as ONE connected digital twin. Do not implement six unrelated mini-projects. A failure in one domain must be able to propagate to relevant downstream entities and trigger a coordinated decision.

#### SIX CONTROL DOMAINS

##### 01 Machine:

- Breakdown
- High temperature
- High vibration
- Production speed drop
- Maintenance required

##### 02 Material:

- Low stock
- Material shortage
- Wrong material
- Delivery delay
- Warehouse full

##### 03 AMR / AGV:

- Low battery
- AGV failure
- AGV stuck
- Congestion
- Blocked route

##### 04 Production:

- Bottleneck
- Production delay
- Urgent order
- Quality defect
- Demand change

#### 05 Factory:

- Power issue
- Network issue
- Emergency
- Environmental abnormality
- Security anomaly

#### 06 Human:

- Worker unavailable
- Task error
- Safety violation
- Manual intervention required

#### DASHBOARD LAYOUT

Create one clean command-center page.

##### LEFT PANEL:

- Show exactly the six domain controls as six expandable cards.
- Each card exposes its relevant event/failure actions.
- Below the six controls, show a live Event Stream.
- Event Stream must include timestamp, domain, severity, target and message.
- Support event filtering.

##### CENTER:

- Large Factory Floor — Live Simulation.
- Render the complete factory using HTML5 Canvas.
- Show storage/raw material, machines, production stations, quality check, finished goods, dispatch, charging stations, corridors, obstacles and restricted/blocked areas.
- Show multiple AGVs/AMRs moving in real time.
- Draw their planned route and destination.
- Visually show blocked routes and congestion.
- Allow click-to-inspect on AGVs and important factory entities.
- Add pause, resume and reset controls.
- Directly below the simulation show AMR/AGV Fleet.
- Fleet summary: Total, Active, Idle, Charging, Failed.
- Fleet table: ID, status, current task, location, battery, speed.

##### RIGHT PANEL:

- AGV Live Auction at the top.
- Show active transport task: pickup, dropoff, priority, deadline and material.
- Show live bids from candidate AGVs.
- Bid columns: AGV, travel cost/time, battery cost/state, congestion cost, future impact, final bid.
- Show selected/winning AGV.
- Below auction show Selected Task Details.
- Show Decision Trace:

SENSE → UNDERSTAND → PREDICT → DECIDE → ACT → UPDATE.

- Below this show System Status and compact live KPIs:  
tasks completed, average task time, fleet utilization, total distance, throughput, congestion events, re-auctions, active alerts.
- Use red/orange primarily for alerts and critical states.

##### VISUAL STYLE

Use a warm, clean, uncluttered interface:

- cream page background
- warm white cards
- brown structure/headings
- restrained orange accents
- red/orange reserved primarily for alerts, emergency and critical failures
- green for healthy/normal state
- generous whitespace
- consistent spacing
- rounded cards
- subtle borders/shadows

- no dark dashboard recreation
- do not copy the provided reference image; use only its information hierarchy
- prioritize readability for a hackathon judge

#### FACTORY SIMULATION

Create a deterministic seeded factory map containing:

- raw material storage
- storage S1 and S2
- Machine-01 through Machine-04
- production stations
- quality inspection
- finished goods
- dispatch
- charging stations C1/C2
- corridors and intersections
- static obstacles
- restricted zones
- at least 7 AGVs/AMRs

Every AGV needs:

id, status, position, heading, battery, speed, currentTask, route, capabilities, lastUpdate.

#### SIMULATION ENGINE

Implement an asynchronous discrete-event/tick loop in FastAPI.

On each tick:

1. Update virtual sensor values.
2. Update machine states and health risk.
3. Update production progress.
4. Consume/transfer inventory.
5. Move AGVs along routes.
6. Update battery.
7. Detect congestion.
8. Detect stuck AGVs.
9. Detect blocked routes.
10. Propagate events.
11. Generate/revise transport tasks.
12. Start/re-run auctions when required.
13. Calculate winning bid.
14. Plan/replan routes using A\*.
15. Apply recovery actions.
16. Recalculate KPIs.
17. Persist important events/history.
18. Broadcast WebSocket updates.

Use deterministic mode with a seed and reset endpoint so the hackathon demo is repeatable.

#### A\* ROUTING

Implement a grid/graph representation of the factory.

A route cost should account for:

- distance
- blocked nodes
- congestion
- battery feasibility
- task priority
- destination feasibility

If a corridor becomes blocked:

- identify affected AGVs
- identify affected tasks
- invalidate affected routes
- calculate alternate routes

- update AGV paths
- update congestion metrics
- add events

#### MARKET-BASED AUCTION ENGINE

Do not simply select the nearest AGV.

For every eligible AGV, calculate:

- travel cost
- battery cost/risk
- current congestion cost
- future-impact congestion cost
- task priority/deadline effect
- feasibility penalties

Produce a transparent final bid score.

Example conceptual model:

final\_bid =

weighted\_travel\_cost

+ weighted\_battery\_cost

+ weighted\_current\_congestion

+ weighted\_future\_impact

+ priority/deadline adjustment

+ infeasibility penalties

Keep the formula configurable in one place.

Store every bid and every component of the bid in SQLite.

The UI must show enough information for a judge to understand why an AGV was selected.

#### FUTURE-IMPACT HEURISTIC

Before selecting an AGV, estimate how assigning the task changes future traffic:

- expected route overlap with other AGVs
- expected congestion at intersections
- expected waiting time
- impact on future task availability
- battery/charging pressure

Use a practical deterministic heuristic; do not pretend it is a trained ML model unless an actual model is implemented.

#### EVENT PROPAGATION

Implement a central event bus/service.

Example:

Machine-03 high vibration

→ machine health risk increases

→ identify affected production orders

→ reduce/redistribute machine workload

→ change material demand

→ create/revise AMR delivery task

→ auction candidates

→ calculate future impact

→ select AGV

→ plan route

→ move AGV

→ update inventory

→ update production

→ update metrics

→ emit Decision Trace events.

The UI must visibly show this chain.

#### FAILURE HANDLERS

Implement practical deterministic handlers for every control domain.

#### Machine:

- breakdown marks machine unavailable
- high temperature increases risk and may reduce capacity
- high vibration increases risk and may reduce workload
- speed drop increases cycle time and creates bottleneck pressure
- maintenance removes machine from new assignments

#### Material:

- low stock triggers replenishment
- shortage identifies affected production
- wrong material blocks/contains the delivery
- delivery delay creates transport risk
- warehouse full redirects storage

#### AMR/AGV:

- low battery may cause charge-before-task or reassignment
- failure removes AGV from active allocation
- stuck triggers recovery/re-route then reassignment
- congestion changes route cost
- blocked route triggers A\* replanning

#### Production:

- bottleneck redistributes work
- delay changes priorities
- urgent order creates high-priority resource demand
- quality defect isolates affected batch
- demand change recalculates material/production/transport requirements

#### Factory:

- power issue puts affected resources into safe simulated state
- network issue prevents unsafe new commands and then resynchronizes
- emergency restricts zones and stops affected movement
- environmental anomaly affects sensitive processes
- security anomaly isolates the simulated affected component and alerts

#### Human:

- worker unavailable causes qualified task reassignment
- task error validates expected vs actual operation
- safety violation triggers restricted-zone response
- manual intervention holds uncertain/high-risk action for operator input

#### INVENTORY

Do not require manual quantity editing during normal operation.

Initial inventory is configured once.

After that, update inventory automatically from:

- production consumption
- AMR transfers
- incoming deliveries
- wrong-material quarantine
- replenishment completion

Use virtual RFID/BLE/sensor events in the prototype.

#### DATABASE

Use SQLite.

Create tables for:

machines

materials

agvs

tasks

auctions

bids

production\_orders

workers

events  
routes  
metrics  
failures

Store timestamps and enough history to calculate:

- average task completion time
- total distance
- throughput
- fleet utilization
- congestion events
- re-auctions
- task success/failure
- inventory changes
- decision history

API

Implement:

GET /api/state  
GET /api/agvs  
GET /api/tasks  
GET /api/events  
GET /api/auctions  
GET /api/metrics  
POST /api/events/inject  
POST /api/tasks  
POST /api/auctions/{taskId}/start  
POST /api/auctions/{auctionId}/bid  
POST /api/controls/{domain}  
POST /api/simulation/reset  
POST /api/simulation/pause  
WS /ws

WEBSOCKET

Use WebSocket for authoritative real-time updates.

Define message types:

STATE\_SNAPSHOT  
AGV\_UPDATE  
MACHINE\_UPDATE  
MATERIAL\_UPDATE  
TASK\_UPDATE  
AUCTION\_UPDATE  
NEW\_EVENT  
ALERT  
DECISION\_TRACE  
METRICS\_UPDATE  
SIMULATION\_STATUS

Send compact updates rather than unnecessarily sending the entire world on every tick.

FRONTEND

Use ES modules and no frontend framework.

Recommended structure:

frontend/  
index.html  
css/  
base.css  
layout.css  
components.css  
simulation.css  
alerts.css

js/  
app.js  
api.js  
websocket.js  
state.js  
controls.js  
events.js  
fleet.js  
auction.js  
decision-trace.js  
metrics.js  
simulation/  
canvas.js  
world.js  
renderer.js  
agv.js  
routes.js  
interactions.js

#### CANVAS

Use a clean coordinate system and scale world coordinates to canvas dimensions.

Use requestAnimationFrame for smooth rendering while receiving authoritative state through WebSocket.

Interpolate movement visually between server ticks, but never change authoritative simulation state in the browser.

#### EVENT STREAM

Every important event should create:

id, timestamp, domain, severity, type, targetId, message.

Newest events appear first.

Critical events are visually prominent but do not flood the screen with red.

#### DECISION TRACE

Display:

SENSE

UNDERSTAND

PREDICT

DECIDE

ACT

UPDATE

When a failure is injected, the current stage should advance as the backend processes it.

#### DEMO MODE

Provide a “Demo Scenario” selector or buttons for:

1. Machine-03 vibration
2. AGV failure
3. Blocked corridor
4. Material shortage
5. Urgent order

Each scenario must be resettable.

The judge should be able to trigger the problem with one action.

The system should automatically perform the recovery.

#### SUCCESS CRITERIA

The prototype is complete only when:

- a failure can be injected with one action
- affected component changes immediately
- a visible risk/status is produced
- at least one downstream impact is identified
- at least one automatic recovery action occurs
- AMR behavior changes when transport is affected

- inventory changes automatically from simulated events
- production metrics update
- cause → decision → action chain is visible
- at least three failure scenarios work reliably

## TESTING

Write tests for:

- A\* routing
- blocked route recovery
- congestion calculation
- battery feasibility
- bid scoring
- future-impact heuristic
- event propagation
- inventory updates
- task reassignment
- deterministic scenario reset

Write integration tests for:

control injection → backend state → auction/routing → WebSocket → UI state.

## ERROR HANDLING

The backend must never crash because of one bad control event.

Return clear HTTP errors.

Log server-side exceptions.

Validate all input.

If the WebSocket disconnects, reconnect automatically and request a fresh state snapshot.

Show a small connection status indicator.

## DELIVERABLES

Produce:

1. Complete project directory.
2. Backend code.
3. Frontend code.
4. SQLite initialization and seed data.
5. Simulation engine.
6. A\* router.
7. Auction engine.
8. Event propagation engine.
9. WebSocket protocol.
10. All six control domains.
11. Factory Canvas renderer.
12. Fleet table.
13. Live auction UI.
14. Decision Trace.
15. KPI calculations.
16. Demo scenarios.
17. Automated tests.
18. README with setup/run instructions.
19. API documentation.
20. A short judge demo guide.

## BUILD METHOD

Build in this order:

Phase 1: skeleton and SQLite

Phase 2: digital twin state

Phase 3: canvas factory

Phase 4: tick loop and AGV movement

Phase 5: A\* and congestion

Phase 6: six controls

Phase 7: event propagation

Phase 8: auction engine

Phase 9: WebSocket

Phase 10: fleet/auction/metrics UI

Phase 11: demo scenarios

Phase 12: testing and polish.

Do not stop at static mockups.

Do not use fake UI buttons that have no backend behavior.

Every visible control must trigger a real state transition.

Every important state transition must be observable in the Event Stream and WebSocket.

The final result must run locally with one documented startup process.

## **END MASTER PROMPT**

## 23. Final Checklist Before Demo

- Run the application from a clean checkout using the README.
- Verify FastAPI starts and WebSocket connects.
- Verify SQLite initializes and seed data loads.
- Verify at least seven AGVs appear and move.
- Verify all six control cards work.
- Inject Machine-03 vibration and observe downstream propagation.
- Fail one AGV and verify re-auction and reassignment.
- Block one corridor and verify A\* rerouting.
- Verify inventory changes automatically.
- Verify the Event Stream records every major decision.
- Verify the auction shows multi-attribute bids.
- Verify the Decision Trace advances through all six stages.
- Reset the simulation and repeat the demo deterministically.

## 24. Judge Explanation — 30 Seconds

“SmartFactory is a live digital twin of a factory. We can inject a disruption such as machine degradation, material shortage, AGV failure or a blocked route. Instead of manually fixing each subsystem, the digital twin updates the complete factory state, predicts the impact, evaluates available AGVs through a multi-attribute market-based auction, selects a recovery action, reroutes resources and updates production and inventory. The dashboard makes the entire SENSE → UNDERSTAND → PREDICT → DECIDE → ACT → UPDATE chain visible.”

Key message: Don’t just automate the factory. Make the factory anticipate, decide and adapt.

## 25. Research Notes Used in the Design

The attached project document is the primary requirements source. External research was used only to support implementation patterns such as digital-twin two-way state exchange, WebSocket real-time dashboards, Canvas-based interactive rendering, normalized live state updates, and API security practices.

Useful references from the research phase include IBM’s digital-twin overview, Konva’s Canvas guidance, real-time dashboard architecture guidance, and OWASP REST security guidance. The locked implementation above intentionally follows the project architecture shown in the supplied System Architecture diagram rather than the earlier exploratory React/Node/MongoDB recommendation.